Todo List — Documentation

Kiran Soodyall
COMS3011A Lab 1: Starting a Project

August 2026

Built with `pdflatex DOCUMENTATION.tex`, run twice so the table of contents resolves. The same content is in `README.md`.

Abstract

A local-first todo application: Next.js for the interface, SQLite for storage. It runs on the user's own machine, serves one user, has no accounts and never talks to a server the user does not control. Tasks carry a title, description, due date and topic, move through three fixed statuses, and are archived rather than deleted. This document covers the third-party code, the database design, how to run it, and how AI was used in building it.

Contents

1 Running It

Node 24 or newer (developed on v24.14.0, npm 11.9.0). The version matters: storage uses Node's built-in `node:sqlite` module, which is only available without a command-line flag from Node 23.4 onwards. Check with `node --version`.

From a clean clone:

```
cd Lab_1/todo_list
npm install
npm run dev
```

Then open `http://localhost:3000`. The database file `todo.db` is created in `Lab_1/todo_list` the first time a page loads, and the schema is applied automatically — there is no separate migration step to run. It is listed in `.gitignore`, so the user's tasks stay theirs.

Stop the server with `Ctrl+C`. Tasks are on disk, so starting it again brings them all back.

To run the tests:

```
npm test
```

To run a production build instead of the dev server:

```
npm run build
npm start
```

Both dev and production print an `ExperimentalWarning` about SQLite on startup. That is Node flagging its own built-in module and is safe to ignore.

1.1 If the application is on an exFAT drive

`next dev` and `next build` both need to create junction points inside `.next`, which exFAT does not support — the result is `failed to create junction point` or `EISDIR: illegal operation on a directory, readlink`. Move the clone to an NTFS volume. Nothing in the application can work around it.

2 Third-Party Code

Nothing here was installed for storage: SQLite comes from Node's built-in `node:sqlite` module, so there is no native `better-sqlite3` build step, no prebuilt binary to download, and nothing to break on a different machine or a non-NTFS drive. The cost is the Node 24 requirement above.

Package	Why
`next`	The framework the brief asks for. Server Components let pages read SQLite directly, and Server Actions handle form posts, so there is no API layer to write or keep in step with the database.
`react`, `react-dom`	Next's rendering layer; not an independent choice.
`typescript`, `@types/node`, `@types/react`, `@types/react-dom`	Types catch the mistakes that matter here — a status string that is not one of the three, a due date that might be null. `@types/node` is pinned to v24 for the `node:sqlite` types.
`tailwindcss`, `@tailwindcss/postcss`, `postcss`, `autoprefixer`	Styling in the markup, so a single-page interface needs no separate stylesheet to keep in sync. Installed by `create-next-app`; kept rather than chosen.
`eslint`, `eslint-config-next`	Catches Next-specific mistakes, such as a Client Component importing server-only code. Also from `create-next-app`.
`vitest`	The test runner. It reads the project's TypeScript and `@/*` path aliases with no extra configuration, and `vitest run` exits when it is done rather than sitting in watch mode.

3 Database Design

One table, in `lib/schema.sql`, applied on first connection by `lib/db.ts`.

3.1 The `tasks` table

Column	Type	Constraints	Holds
`id`	INTEGER	primary key, autoincrement	Stable identifier, and the `<id>` in a task's edit route.
`title`	TEXT	not null, `length(trim(title)) > 0`	The task. A whitespace-only title is rejected by the database, not just the form.
`description`	TEXT	not null, default `''`	Optional detail, stored as an empty string rather than null so reads need no null check.
`due_date`	TEXT	nullable	`YYYY-MM-DD`. Null means no due date, which is why the column is nullable.
`topic`	TEXT	not null, default `'General'`	Free text, entered per task.
`status`	TEXT	not null, default `'todo'`, `CHECK (status IN ('todo', 'in_progress', 'complete'))`	One of the three fixed statuses.
`archived_at`	TEXT	nullable	Null while active; a timestamp once archived.
`created_at`	TEXT	not null, default `datetime('now')`	Set on insert, never updated.
`updated_at`	TEXT	not null, default `datetime('now')`	Bumped by every edit, archive and restore.

Two indexes: `idx_tasks_archived` on `archived_at`, because every list query filters on it, and `idx_tasks_due_date` on `due_date`, the default sort.

3.2 Relationships

There are none, deliberately. Topic and status are both single values belonging to one task, so each is a column on `tasks` rather than a row in its own table joined back:

- **Status** is fixed at three values that the brief says are not user-customisable. A `statuses` table would let the database hold a fourth one; a `CHECK` constraint cannot. The constraint is the stricter model, and it rejects `'overdue'` as a status on both insert and update.
- **Topic** is a label the user types per task, with no attributes of its own. A `topics` table would add a foreign key and a join for no gain, and would raise a question the brief never asks — what happens to a topic when its last task is archived. Grouping by topic is a sort on the column.

3.3 Two things are deliberately not stored

Archive is a flag, not a move. `archiveTask` sets `archived_at` and nothing else; the row does not move and no row is ever deleted, which is what keeps an archived task viewable. The active list is `WHERE archived_at IS NULL` and the archive is `WHERE archived_at IS NOT NULL`. There is no `deleteTask` function anywhere in the codebase, and restoring is the same operation in reverse.

Overdue is derived, not a column. A stored flag would be wrong the moment midnight passed, and a stored status would make overdue a fourth status, which the brief forbids. `isOverdue` in `lib/tasks.ts` works it out at read time: a task is overdue when its due date is before today and it is neither complete nor archived. Due dates are whole days, so the comparison is between `YYYY-MM-DD` strings and today's local date — reading the due date into a `Date` treats it as UTC midnight, which flags a task due *today* as overdue from 02:00 local time onwards.

4 How It Fits Together

```
app/layout.tsx            shell, fonts, and the jump scare timer
app/page.tsx              active list, sort controls, new task form
app/archive/page.tsx      archived tasks, with restore
app/tasks/[id]/edit/      edit form for one task
app/actions.ts            server actions: create, update, archive, restore
app/components/           TaskForm.tsx, TaskList.tsx (cards + SortBar),
                          JumpScare.tsx
lib/tasks.ts              queries, sort clauses, the overdue rule
lib/db.ts                 opens the database, applies the schema
lib/schema.sql            the schema above
tests/tasks.test.ts       the suite npm test runs
```

Pages read the database directly through `lib/tasks.ts` and forms post to server actions, so there are no API routes and no client-side data fetching. Sorting is held in the URL (`/?sort=topic`) rather than component state, which keeps the ordering in SQL and lets a sorted list survive a reload.

5 Testing

`npm test` runs `tests/tasks.test.ts` under Vitest: twelve tests covering creation, editing, archiving and restoring, the overdue rule, and all three sort orders. They never touch the developer's own database — `TODO_DB_PATH` is pointed at a `mkdtemp` file before the data layer is imported, tables are cleared before each test, and the temporary directory is removed at the end.

The overdue cases run against a fixed clock (11 August 2026, 22:30) rather than the current time, so they do not depend on when the suite is run. Two of the tests were checked by deliberately reintroducing the bugs they were written for: restoring the `new Date(row.due_date) < now` comparison fails the "due today is not overdue" case, and restoring an alphabetical `ORDER BY status` fails the workflow-order case.

6 Jump Scares

Once every 60 seconds the application covers the screen with one of five gifs, picked at random, plays `public/scare/scare_audio.mp3` over it, and clears after 2.5 seconds. It is implemented in `app/components/JumpScare.tsx` and mounted in `app/layout.tsx`, so it runs on every page.

It has nothing to do with the todo list and nothing to do with the brief. The overlay is `aria-hidden` and `pointer-events-none`, so it does not interrupt a form or reach a screen reader, and the gif is chosen when the timer fires rather than during render, so server and client cannot disagree on hydration. Browsers block audio until the page has been clicked, so the first scare of a session may be silent.

This feature was written in full by Claude Opus 5 (Claude Code). No part of it was written by hand. The author's prompt, verbatim:

> lastly every minute you will create a jump scare using the five gifs i added and i included the audio. Add this to the documentation that you as the ai fully implemented it as well. Add the prompt i used as well

The five gifs and the audio file were supplied by the author; the AI moved them from `scare_files/` into `public/scare/` so that Next.js could serve them, wrote the component, mounted it in the layout, and verified that all six assets return 200 and that the timer, `new Audio`, the random pick and both intervals appear in the client bundle. The 60-second firing itself was not verified in a browser.

7 AI Usage

The application was built with Claude Opus 5 (Claude Code) under constraints set by the author up front: build the lab incrementally so the commit history reads as human work, split each feature into a backend commit and a frontend commit, plant minor deliberate faults to be fixed in later commits, and make no commits without the author's say-so. Full transcripts accompany this submission.

Two occasions where the author's or the tool's output was rejected and redirected are worth recording, because both changed the shipped code:

- **The author rejected the AI's first working slice as too large.** The AI had written the schema, data layer, server actions, all pages, the test suite and the documentation in a single pass. The author stopped it ("you are doing too much work for 1 commit"), and the work was pulled back out of the tree and re-added feature by feature, two commits per feature.
- **The AI's first storage choice did not work and was replaced.** `better-sqlite3` was installed first, as the conventional choice. It crashed both `next dev` and `next build`: the project lives on an exFAT volume, which has no reparse points, so Turbopack could not create the junction it needs to load a native module. The fallback to webpack failed separately. The dependency was removed in favour of Node's built-in `node:sqlite`, which needs no native build, and the author chose that route over relocating the repository.

Deliberate faults were planted and then fixed in later commits, each fix its own commit: a missing `revalidatePath` that left a newly created task invisible until reload; an edit action with no redirect, leaving the user on a dead-end form; an alphabetical status sort that put completed work at the top; and the UTC due-date comparison described in section 3.3, which flagged tasks due today as overdue. Each was demonstrated against a running server before being fixed.